

Pre-emptible kernel and Non-preemptible kernel

- Pre-emptible kernel
 - User space and kernel space preemption of process is supported.
 - Better responsiveness
 - Real time systems
- Non Pre-emptible kernel
 - User space preemption of process is supported.
 - Better throughput
 - Server systems and heavy computations

RTOS types

- RTOS can be classified as follows:
 - Embedded RTOS
 - Real-time Linux
 - Linux based RTOS -- Dual kernel approach

Real-time Linux

- Linux kernel can be configured with RT settings E.g. HZ=1000, Kernel=Pre-emptible Kernel, etc.
- Linux kernel source code can be modified heavily to provide realtime behaviour. E.g. scheduler = O(1), interrupt handling mechanism updated for minimal latency.
- "RTPatch" is an open-source project (rtpatch.org) by Ingo Monlar that converts Linux into RTOS.

Dual kernel approach

- Linux kernel is treated as lowest priority task by the "real time executive/kernel". Hence when no real time tasks are running, Linux kernel is scheduled. Then Linux scheduler share the available CPU time with Linux processes.
- Hardware interrupts in this approach are controlled/managed by the real time executive. They will be dispatched to the Linux kernel only if not handled by the RT tasks. The RT kernel act as "virtual PIC" for the Linux kernel. When Linux kernel try to disable/mask the interrupt, RT kernel will not send interrupts to the Linux kernel (Note that hardware interrupts cannot be manipulated by Linux kernel directly).
- The RT executive mainly contains RT task management, RT scheduler and RT IPC mechanisms. The rest of the functionality i.e. memory management, file io, networking, etc will be handled by the Linux kernel. Due to this size of RT kernel is too small & it also referred as "nano-kernel".
- E.g. RTAI (Real-time Application Interface), Xenomai, etc.

Embedded RTOS

- Also known as Traditional RTOS or Conventional RTOS.
- RTOS is embedded into embedded controller device along with real-time application/tasks.
 - RTOS kernel + RT Tasks ---> .hex or .bin ---> uC Flash
- ARM application development
 - Source code (C/Asm) --> Cross Compile --> Executable (ELF) --> Hex/Bin file --> ARM uC Flash
 - Application have full control of Hardware peripherals, Timers, Interrupts, etc.
 - Difficult to manage (modularize, debug, ...) in complex applications (multiple peripherals).

- ARM + Embedded RTOS application development
 - Source code + RTOS code (C/Asm) --> Cross Compile --> Executable (ELF) --> Hex/Bin file --> ARM uC Flash
 - RTOS reserve some system peripherals e.g. Timer, Interrupt controller, Software Interrupt, etc.
 - Application code can be divided into RT tasks (e.g. LED task, ADC task, UART task, etc). These task may communicate with each other.
 - E.g. FreeRTOS, uC-OS, etc

RTOS Tasks

- Three types of tasks
 - Aperiodic task
 - Periodic task
 - Sporadic task

Aperiodic task

- Also called as "one-shot task".
- Executed only once -- Need CPU time only once.
- Task may re-run (need CPU time again) after a random interval (can be zero).
- Timing requirement: $0 < t \leq d$
 - t = Execution time
 - d = Deadline

```
void my_task1(...) {
    // some work -- may need 1 to 5 ms.
}
// time = 4
```

```
void my_task2(...) {
    for(i = 1; i <= 5; i++) {
        // some work -- may need 1 to 5 ms.
        delay(10 ms); // delay can be for random time or even zero time.
    }
}
// time = 2 + 10 + 4 + 10 + 1 + 10 + 3 + 10 + 2 + 10 = 60
```

Periodic task

- Needs CPU time periodically (fixed time).
- Task cycle is repeated after a fixed time. Task release points are fixed.
- Timing requirement: $0 < t \leq d \leq p$
 - t = Execution time
 - d = Deadline
 - p = Period -- Time after which CPU time is to be reallocated to the task.
- If deadline (d) is not mentioned, then consider it same as period (p).

- CPU utilization for each task = t / p
- CPU utilization for all tasks = $(t_1 / p_1) + (t_2 / p_2) + (t_3 / p_3) + \dots$

```
void my_task1(...) {
    for(i = 0; i <= 5; i++) {
        // some work -- may need 1 to 5 ms.
        delay_until(10 ms); // period = 10 ms // wait time = period - actual
task time
    }
}
// time = 2 + 8 + 4 + 6 + 1 + 9 + 3 + 7 + 2 + 8 = 50 ms.
```

Sporadic task

- Similar to Aperiodic task.
- Task will need CPU time after some random interval. The time interval must not be zero.
- Timing requirement: $(0 < t \leq d) + w$
 - t = Execution time
 - d = Deadline
 - w = Wait for some time ($w > 0$)

```
void my_task1(...) {
    for(i = 0; i <= 5; i++) {
        // some work -- may need 1 to 5 ms.
        delay_until(random event); // delay must not be zero.
    }
}
```

Task Functions

```
void ATaskFunction( void *pvParameters );
```

- Each task is a small program in its own right. It has an entry point, will normally run forever within an infinite loop, and will not exit.

TaskHandle_t

- Type by which tasks are referenced
- a call to `xTaskCreate` returns (via a pointer parameter) an `TaskHandle_t` variable that can then be used as a parameter to `vTaskDelete` to delete the task

xTaskCreate

```

 BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,
                        const char * const pcName,
                        const configSTACK_DEPTH_TYPE uxStackDepth,
                        void *pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t *pxCreatedTask
                        );

```

- Create a new task and add it to the list of tasks that are ready to run
- Parameters:
 - pvTaskCode
 - Pointer to the task entry function (just the name of the function that implements the task)
 - Tasks are normally implemented as an infinite loop; the function which implements the task must never attempt to return or exit.
 - Tasks can, however, delete themselves.
 - pcName
 - A descriptive name for the task.
 - This is mainly used to facilitate debugging, but can also be used to obtain a task handle.
 - uxStackDepth
 - The number of words (not bytes!) to allocate for use as the task's stack.
 - pvParameters
 - A value that is passed as the parameter to the created task
 - uxPriority
 - The priority at which the created task will execute
 - Priorities can be assigned from 0, which is the lowest priority, to (configMAX_PRIORITIES – 1), which is the highest priority.
 - pxCreatedTask
 - Used to pass a handle to the created task out of the xTaskCreate() function.
 - pxCreatedTask is optional and can be set to NULL.
- Returns:
 - If the task was created successfully then pdPASS is returned.
 - Otherwise errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY is returned.

vTaskDelete

```

void vTaskDelete( TaskHandle_t xTask );

```

- Remove a task from the RTOS kernels management
- Parameters:
 - xTask
 - The handle of the task to be deleted. Passing NULL will cause the calling task to be deleted.

vTaskDelay

```
void vTaskDelay( const TickType_t xTicksToDelay );
```

- Delay a task for a given number of ticks.
- The actual time that the task remains blocked depends on the tick rate.
- Parameters:
 - xTicksToDelay
 - The amount of time, in tick periods, that the calling task should block.
 - For example, if a task called vTaskDelay(100) while the tick count was 10,000, then it would immediately enter the Blocked state and remain there until the tick count reached 10,100.
 - The constant portTICK_RATE_MS can be used to convert milliseconds into ticks.

vTaskDelayUntil

```
void vTaskDelayUntil( TickType_t *pxPreviousWakeTime,
                     const TickType_t xTimeIncrement );
```

- Delay a task until a specified time.
- This function can be used by periodic tasks to ensure a constant execution frequency.
- Parameters:
 - pxPreviousWakeTime
 - Pointer to a variable that holds the time at which the task was last unblocked.
 - The variable must be initialised with the current time prior to its first use (see the example below).
 - Following this the variable is automatically updated within vTaskDelayUntil()
 - xTimeIncrement
 - The cycle time period

xTaskDelayUntil

```
BaseType_t xTaskDelayUntil( TickType_t *pxPreviousWakeTime,
                           const TickType_t xTimeIncrement );
```

- Parameters:
 - pxPreviousWakeTime
 - Pointer to a variable that holds the time at which the task was last unblocked.
 - The variable must be initialised with the current time prior to its first use (see the example below).
 - Following this the variable is automatically updated within vTaskDelayUntil()
 - xTimeIncrement
 - The cycle time period
- Returns:

- A value which can be used to check whether the task was actually delayed: pdTRUE if the task was delayed and pdFALSE otherwise.

vTaskStartScheduler

```
void vTaskStartScheduler( void );
```

- Starts the RTOS scheduler

SUNBEAM